

Open in app ↗



Search

Write

Welcome Offer Access to everything. Now 30% off. [Upgrade now](#)

Text Preprocessing in NLP: Bag of Words (BoW) and TF-IDF



Marishetty Ramyakrishna 12 min read · 8 hours ago



Natural Language Processing (NLP) allows computers to understand and work with human language.

We use NLP in many real-world applications such as:

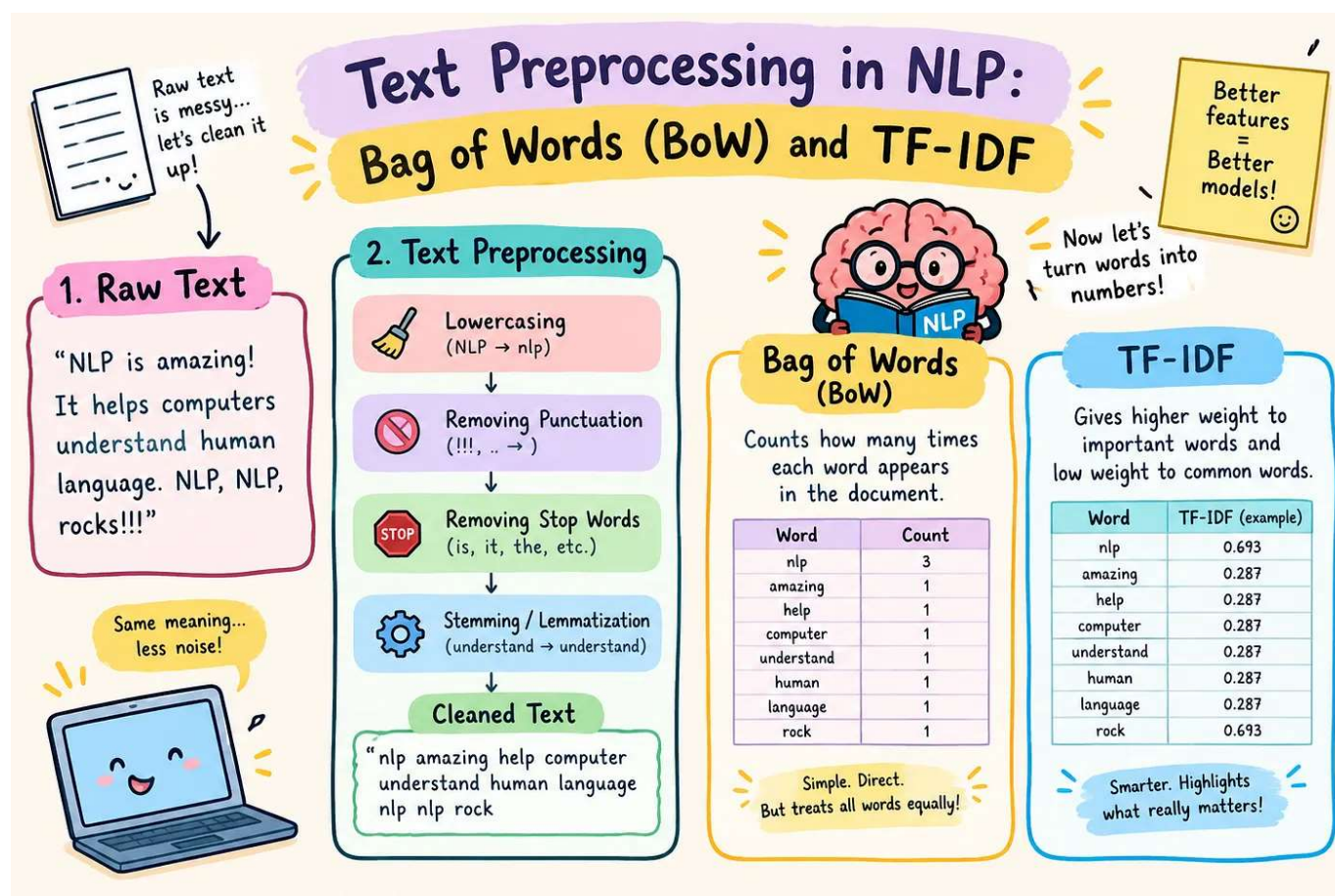
- Chatbots
- Sentiment Analysis
- Spam Detection
- Text Classification
- Recommendation Systems
- Search Engines
- Email Classification
- Document Classification

However, Machine Learning models cannot directly understand raw text such as:

“I love learning Machine Learning.”

Computers need numerical representations of text before a Machine Learning model can process it.

This is where Text Preprocessing and Text Vectorization become important.



In this article, we will understand two important techniques used to convert text into numerical form:

1. Bag of Words (BoW)

2. TF-IDF

We will understand their concepts, working, mathematical formulas, Python implementation, advantages, limitations, and differences.

What is NLP?

Natural Language Processing (NLP) is a field of Artificial Intelligence that focuses on enabling computers to process, understand, and work with human language.

For example, consider the sentence:

```
```text
```

“I love Machine Learning.”

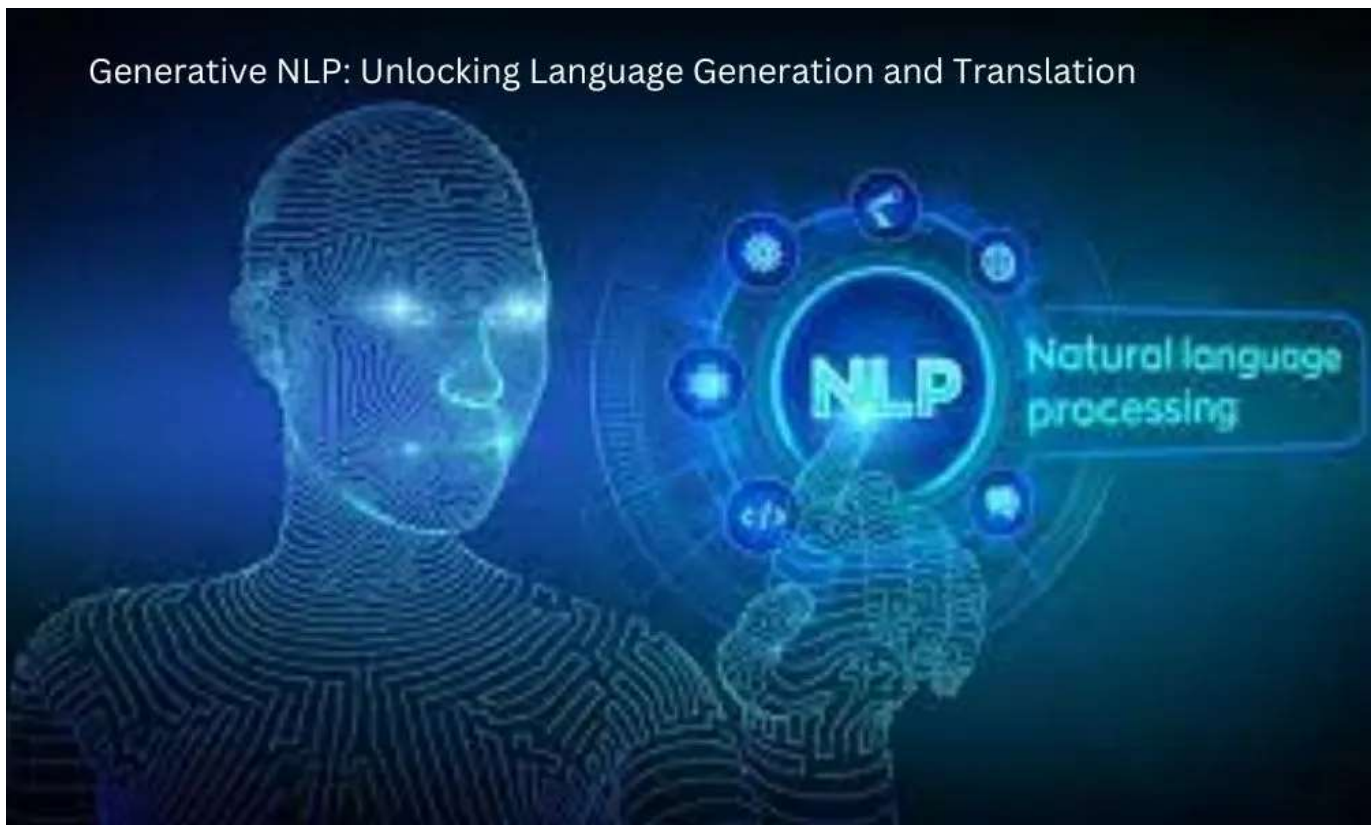
Humans can easily understand this sentence.

But a Machine Learning algorithm cannot directly understand the meaning of words such as:

```
love
Machine
Learning
```

Therefore, we need to convert text into numerical representations.

This process is called **Text Vectorization**.



## What is Text Preprocessing?

Text preprocessing is the process of cleaning and preparing raw text before using it for NLP or Machine Learning tasks.

Raw text may contain:

- Uppercase and lowercase letters
- Punctuation
- Stop words
- Special characters
- Extra spaces
- Numbers
- Different word forms

For example:

```
"HELLO!!! I am learning NLP."
```

After preprocessing, it may become:

```
"hello learning nlp"
```

The exact preprocessing steps depend on the NLP task and the model being used.

## Common NLP Text Preprocessing Steps

A traditional NLP preprocessing workflow may include:

```
Raw Text
 ↓
Lowercasing
 ↓
Remove Punctuation
 ↓
Tokenization
 ↓
Remove Stop Words
 ↓
Stemming / Lemmatization
 ↓
Text Vectorization
 ↓
Machine Learning Model
```

Let's briefly understand these steps.

## Lowercasing

We convert all text into lowercase.

For example:

```
"Machine Learning Is Amazing"
```

becomes:

```
"machine learning is amazing"
```

This prevents the model from treating:

```
Machine
machine
MACHINE
```

as completely different words.

Python:

```
text = "Machine Learning Is Amazing"
```

```
text = text.lower()
print(text)
```

Output:

```
machine learning is amazing
```

## Removing Punctuation

Punctuation marks such as:

```
.
,
!
?
;
:
```

can often be removed depending on the task.

Example:

"Hello, how are you?"

After removing punctuation:

"Hello how are you"

Python:

```
import string
```

```
text = "Hello, how are you?"
text = text.translate(
 str.maketrans('', '', string.punctuation)
)
print(text)
```

Output:

Hello how are you

## Tokenization

**Tokenization** is the process of breaking text into smaller units called tokens.

For example:

```
"I love NLP"
```

can be tokenized into:

```
["I", "love", "NLP"]
```

Each word becomes an individual token.

Tokenization is an important step in NLP because many NLP techniques operate on tokens.

## Stop Words

Stop words are commonly occurring words that may not carry much useful information for certain NLP tasks.

Examples include:

```
the
is
a
an
and
of
```

```
in
to
```

For example:

```
"The cat is sitting on the table."
```

After removing some stop words:

```
"cat sitting table"
```

However, stop-word removal is **not always necessary**.

For some NLP tasks, words such as:

```
not
no
never
```

can be extremely important.

So stop words should be removed only when appropriate for the task.

## Stemming

Stemming reduces words to a simpler root-like form.

For example:

```
playing
played
plays
```

may be reduced to:

```
play
```

However, stemming can sometimes produce words that are not grammatically correct.

## Lemmatization

Lemmatization converts words into their meaningful base form.

For example:

```
running → run
better → good
studies → study
```

Lemmatization generally tries to produce a valid dictionary form of the word.

## What is Text Vectorization?

After preprocessing, we still have text.

Machine Learning algorithms need numerical input.

Therefore, we need to convert text into numbers.

This process is called:

### Text Vectorization

For example:

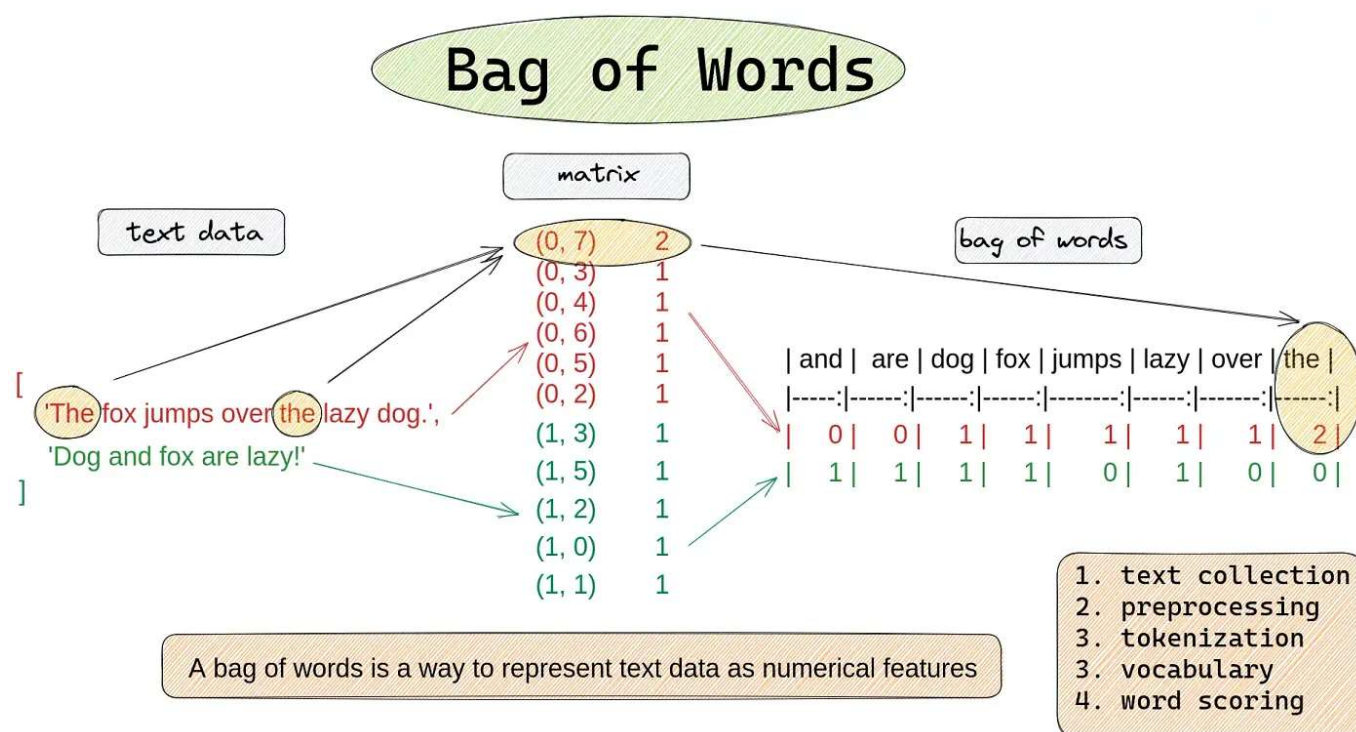
```
graph TD; A[Text] --> B[Preprocessing]; B --> C[Vectorization]; C --> D[Numerical Features]; D --> E[Machine Learning Model];
```

Text  
↓  
Preprocessing  
↓  
Vectorization  
↓  
Numerical Features  
↓  
Machine Learning Model

Two classical and important vectorization techniques are:

- Bag of Words (BoW)
- TF-IDF

## Part 1: Bag of Words (BoW)



## What is Bag of Words?

Bag of Words (BoW) is a simple technique for converting text into numerical vectors based on word frequency.

The main idea is:

Treat the text as a collection, or “bag,” of words and count how frequently each word appears.

The order of words is ignored.

## Simple Example of Bag of Words

Suppose we have three sentences:

Document 1: I love Machine Learning

Document 2: I love Python

Document 3: Machine Learning is amazing

First, we create a vocabulary containing all unique words:

I  
love  
Machine  
Learning  
Python  
is  
amazing

Now we count how many times each word appears in each document.

The result may look like this:

Document	Binary Representation
<b>Document 1</b> I love Machine Learning	11111000
<b>Document 2</b> Python is amazing	11001000
<b>Document 3</b>	00110111

Now the text has been converted into numbers.

This numerical representation can be given to a Machine Learning model.

## How Does Bag of Words Work?

The basic process is:

```
Documents
 ↓
Tokenization
 ↓
Create Vocabulary
 ↓
Count Word Occurrences
```

↓  
Create Numerical Vectors

Let's understand each step.

## Step 1: Collect the Documents

Suppose:

```
documents = [
 "I love machine learning",
 "I love Python",
 "machine learning is amazing"
]
```

## Step 2: Create the Vocabulary

The unique words are:

```
I
love
machine
learning
Python
is
amazing
```

This becomes our vocabulary.

## Step 3: Count the Words

For each document, we count how many times each vocabulary word appears.

For example:

```
"I love machine learning"
```

becomes:

```
I → 1
love → 1
machine → 1
learning → 1
Python → 0
is → 0
amazing → 0
```

This produces a numerical vector.

## Bag of Words using Python

Scikit-learn provides `CountVectorizer` for implementing Bag of Words.

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
documents = [
 "I love machine learning",
 "I love Python",
```

```
"machine learning is amazing"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)
print(X.toarray())
```

Output will be a numerical matrix representing word counts.

## Viewing the Vocabulary

We can also see the words learned by the vectorizer.

```
print(vectorizer.get_feature_names_out())
```

The output will contain the vocabulary created from the documents.

## Understanding `fit_transform()`

In:

```
X = vectorizer.fit_transform(documents)
```

`fit()` learns the vocabulary from the documents.

`transform()` converts the documents into numerical vectors using that vocabulary.

Therefore:

```
fit()
↓
Learn Vocabulary
```

```
transform()
↓
Convert Text into Numerical Vectors
```

`fit_transform()` performs both operations together.

## Example with Repeated Words

Consider:

```
Document 1:
machine learning machine
```

The word:

```
machine
```

appears twice.

Therefore, its Bag of Words value will be:

```
machine → 2
```

while:

```
learning → 1
```

So BoW represents the document based on word frequency.

## Important Limitation of Bag of Words

Bag of Words ignores the order of words.

Consider:

```
"I love Python"
```

```
"Python love I"
```

BoW may represent both sentences similarly because they contain the same words.

Therefore, BoW does not understand:

- Word order
- Grammar
- Context
- Meaning

It mainly focuses on word occurrence.

## Another Limitation: Common Words

Suppose we have thousands of documents.

Words such as:

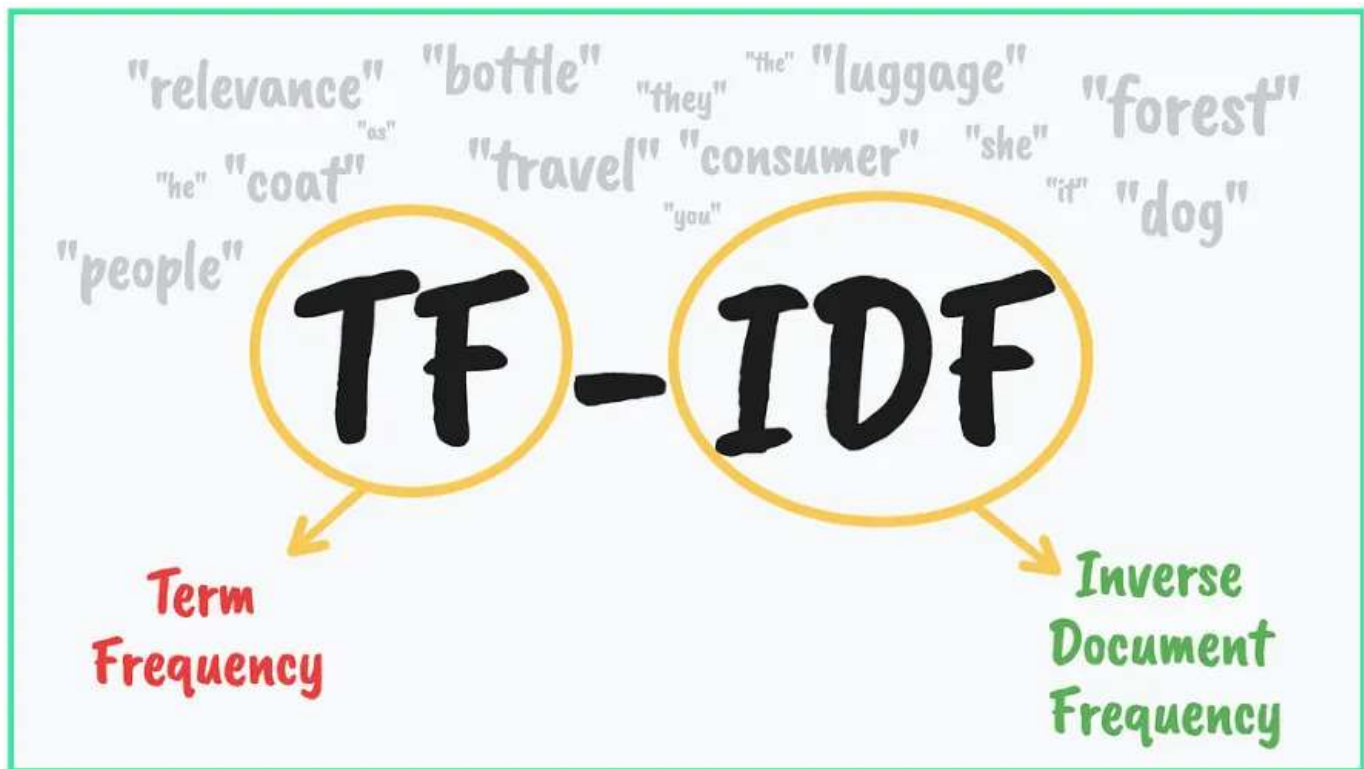
the  
is  
and  
of

may appear very frequently.

BoW gives these words high counts even if they do not provide much information for distinguishing documents.

This is one reason why another technique, **TF-IDF**, is useful.

## Part 2: TF-IDF



## What is TF-IDF?

TF-IDF stands for:

**Term Frequency — Inverse Document Frequency**

It is a numerical representation that measures how important a word is to a document relative to a collection of documents.

The main idea is:

A word is important if it appears frequently in a particular document but does not appear in too many other documents.

## Why Do We Need TF-IDF?

Consider these two words:

machine  
the

Suppose:

"machine"

appears frequently in one particular document.

It may be useful for understanding what that document is about.

But:

"the"

may appear in almost every document.

Therefore, “the” provides less information for distinguishing one document from another.

TF-IDF tries to give:

**Higher weight → More informative words**

**Lower weight → Common words**

## **TF-IDF Has Two Main Components**

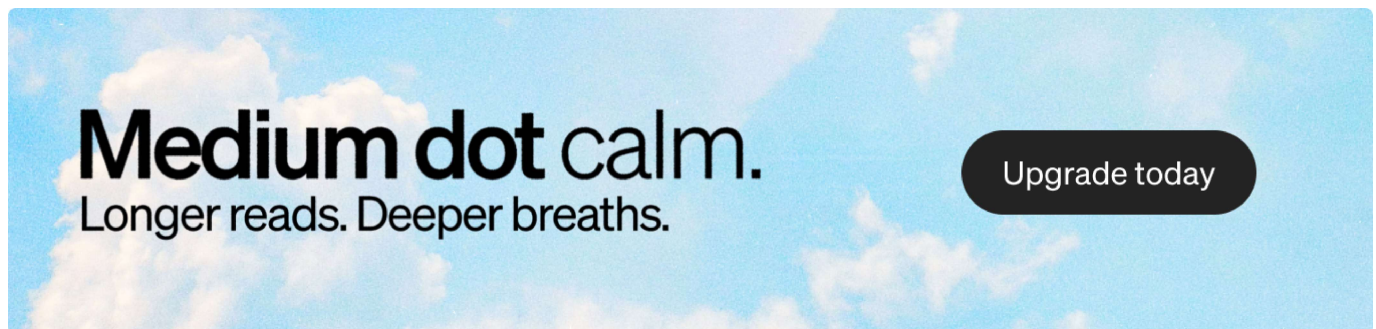
TF-IDF consists of:

1. TF → Term Frequency
2. IDF → Inverse Document Frequency

The final score is based on both.

### **Term Frequency (TF)**

Term Frequency measures how frequently a word appears in a particular document.



A common formula is:

**TF = Number of times the term appears in the document / Total number of terms in the document**

### **Example of Term Frequency**

Suppose our document contains:

machine learning machine

Total words:

3

The word “machine” appears:

2 times

Therefore:

$$\text{TF}(\text{machine}) = 2 / 3$$

So:

$$\text{TF}(\text{machine}) = 0.67$$

The word “learning” appears once:

$$\text{TF}(\text{learning}) = 1 / 3$$

So:

$$\text{TF}(\text{learning}) = 0.33$$

## Inverse Document Frequency (IDF)

IDF measures how rare or common a word is across a collection of documents.

A commonly used form is:

$$\text{IDF} = \log(N / \text{df})$$

Where:

- $N$  = Total number of documents
- $\text{df}$  = Number of documents containing the term

The exact formula used in software libraries can include smoothing, but the basic idea remains the same.

## Understanding IDF

Suppose we have:

100 documents

A word appears in:

90 documents

That word is very common.

Therefore, its IDF will be relatively low.

Now suppose another word appears in:

5 documents

That word is much rarer.

Therefore, its IDF will be higher.

So:

Common Word



Lower IDF

Rare Word



Higher IDF

## TF-IDF Formula

The basic formula is:

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

So a word receives a high TF-IDF score when:

- It appears frequently in the current document
- It is relatively uncommon across the entire document collection

## Simple Intuition

Think of TF-IDF like this:

Word appears frequently in this document  
+  
Word is uncommon across other documents  
↓  
High TF-IDF

But:

Word appears everywhere  
↓  
Low TF-IDF

This helps highlight words that are more useful for distinguishing documents.

## TF-IDF using Python

Scikit-learn provides `TfidfVectorizer`.

```
from sklearn.feature_extraction.text import TfidfVectorizer

documents = [
 "I love machine learning",
 "I love Python",
 "machine learning is amazing"
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)
print(X.toarray())
```

This converts the documents into TF-IDF numerical vectors.

## Viewing the Vocabulary

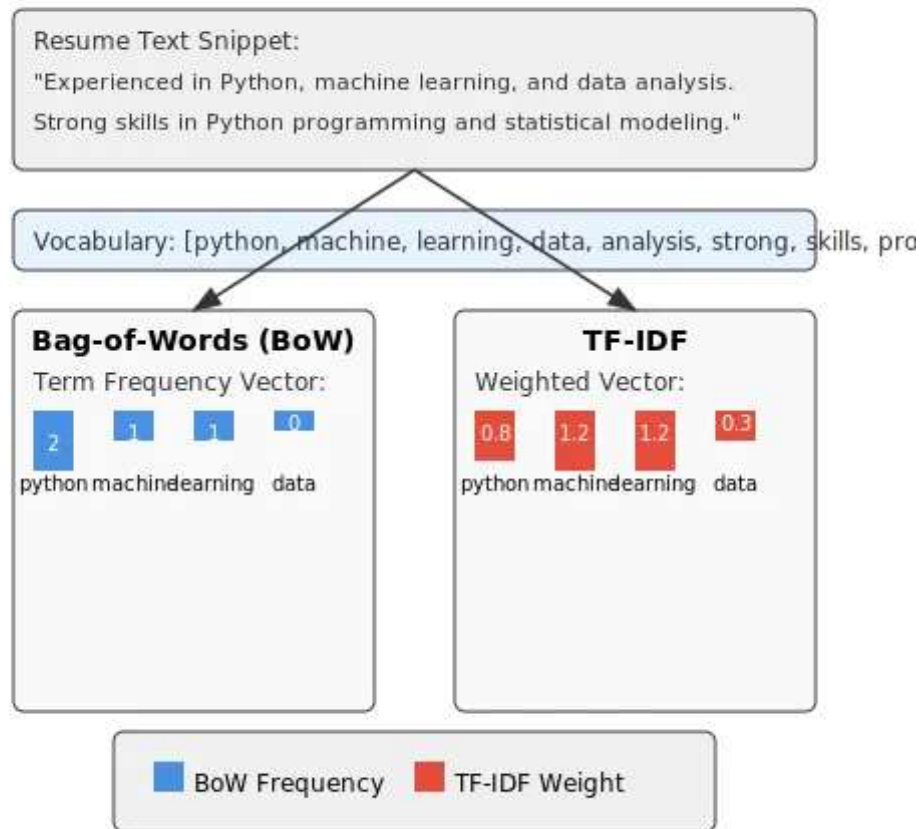
We can view the vocabulary using:

```
print(vectorizer.get_feature_names_out())
```

This shows the words used as features.

## BoW vs TF-IDF

## BoW vs. TF-IDF Vector Representation



### Which One Should You Use?

There is no single technique that is always best.

The choice depends on:

- Dataset size
- NLP task
- Number of features
- Model
- Frequency of common words
- Desired interpretability

## Use Bag of Words when:

- You need a simple representation
- Word counts are useful
- You want a straightforward baseline
- You are learning basic text vectorization

## Use TF-IDF when:

- Common words should receive less importance
- You want to emphasize informative words
- You are working with document classification
- You want a strong traditional NLP baseline

## Using BoW with a Machine Learning Model

Let's see how text can be converted into numbers and then used with a Machine Learning model.

Suppose we want to classify messages as:

Spam  
Not Spam

Our data could look like:

```
"I won a free prize"
"Meeting is scheduled tomorrow"
"Congratulations you won money"
"Please attend the meeting"
```

First, we convert the text into numerical features using BoW.

Then we can train a Machine Learning model.

The workflow becomes:

```
Text Data
 ↓
Text Preprocessing
 ↓
Bag of Words
 ↓
Numerical Features
 ↓
Machine Learning Model
 ↓
Prediction
```

## Simple BoW + Machine Learning Example

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
```

```
documents = [
 "I won a free prize",
```

```
"Meeting is scheduled tomorrow",
"Congratulations you won money",
"Please attend the meeting"

]

labels = [
 "Spam",
 "Not Spam",
 "Spam",
 "Not Spam"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)

model = LogisticRegression()
model.fit(X, labels)

new_message = ["You won a free prize"]
new_message_vector = vectorizer.transform(new_message)
prediction = model.predict(new_message_vector)
print(prediction)
```

The important idea is:

```
Text
↓
Vectorization
↓
Numerical Matrix
↓
Machine Learning Model
↓
Prediction
```

## TF-IDF + Machine Learning Example

We can replace Bag of Words with TF-IDF.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
```

```
documents = [
 "I won a free prize",
 "Meeting is scheduled tomorrow",
 "Congratulations you won money",
 "Please attend the meeting"
]

labels = [
 "Spam",
 "Not Spam",
 "Spam",
 "Not Spam"
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)
model = LogisticRegression()
model.fit(X, labels)
new_message = ["You won a free prize"]
new_message_vector = vectorizer.transform(new_message)
prediction = model.predict(new_message_vector)
print(prediction)
```

Here, TF-IDF converts the text into weighted numerical features before sending them to the Machine Learning model.

## Important: Train-Test Split with Text Data

When working with a real Machine Learning project, we should split the dataset into training and testing data.

For example:

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
 documents,
 labels,
 test_size=0.2,
 random_state=42
)
```

Then the vectorizer should be fitted only on the training text.

Correct approach:

```
vectorizer.fit(X_train)
```

```
X_train_vectorized = vectorizer.transform(X_train)
```

```
X_test_vectorized = vectorizer.transform(X_test)
```

Or:

```
X_train_vectorized = vectorizer.fit_transform(X_train)
```

```
X_test_vectorized = vectorizer.transform(X_test)
```

Remember:

Training Data → `fit_transform()`

Testing Data → `transform()`

Lowercasing

We convert all text into lowercase.

For example:

"Machine Learning Is Amazing"

becomes:

"machine learning is amazing"

This prevents the model from treating:

Machine  
machine  
MACHINE

Lowercasing

We convert all text into lowercase.

For example:

"Machine Learning Is Amazing"

becomes:

"machine learning is amazing"

This prevents the model from treating:

Machine  
machine  
MACHINE

as completely different words.as completely different words.

This prevents information from the test set from in

## Data Leakage in NLP

Data leakage can also happen during text preprocessing.

For example, this is not the correct approach:

```
vectorizer.fit_transform(all_documents)
```

before splitting the data.

Why?

Because the vectorizer learns information from the entire dataset, including the test set.

Instead:

```
Split Data
 ↓
Training Text
 ↓
Fit Vectorizer
 ↓
Transform Training Data
 ↓
Transform Test Data
```

This is an important Machine Learning practice.

## Limitations of BoW and TF-IDF

Although BoW and TF-IDF are useful, they have limitations.

### 1. No Word Order

They do not understand word order.

For example:

```
"I love Python"
```

and:

```
"Python love I"
```

can have similar representations.

### 2. No Deep Semantic Understanding

BoW and TF-IDF do not truly understand the meaning of words.

For example:

"car"

and:

"automobile"

may be treated as completely different features even though they have similar meanings.

### 3. Large Vocabulary

If the dataset contains thousands or millions of unique words, the resulting feature matrix can become very large.

This can lead to:

- High memory usage
- High computational cost
- Sparse matrices

### 4. Context is Not Captured

Consider:

"I like this movie"

and:

"I do not like this movie"

The presence of the word “like” alone does not capture the important role of “not”.

Traditional BoW and TF-IDF have limited ability to understand such context.

## What Came After BoW and TF-IDF?

BoW and TF-IDF are traditional NLP techniques.

Modern NLP also uses more advanced representations such as:

- Word2Vec
- GloVe
- FastText
- Word Embeddings
- Transformers
- BERT
- Large Language Models

These approaches can capture more information about relationships between words and context.

However, BoW and TF-IDF are still valuable because they are:

- Simple
- Fast
- Easy to understand
- Easy to implement
- Useful as baseline approaches
- Commonly used in traditional NLP Machine Learning tasks

Understanding BoW and TF-IDF also helps build a strong foundation before learning more advanced NLP techniques.

## Complete NLP Workflow

A traditional NLP Machine Learning workflow can look like this:

```
graph TD; A[Raw Text] --> B[Text Cleaning]; B --> C[Lowercasing]; C --> D[Tokenization]; D --> E[Stop Word Handling]; E --> F[Stemming / Lemmatization]; F --> G[Train-Test Split];
```

Raw Text  
↓  
Text Cleaning  
↓  
Lowercasing  
↓  
Tokenization  
↓  
Stop Word Handling  
↓  
Stemming / Lemmatization  
↓  
Train-Test Split

```
↓
Text Vectorization
↓
BoW / TF-IDF
↓
Numerical Features
↓
Machine Learning Model
↓
Prediction
↓
Evaluation
```

The exact preprocessing steps depend on the dataset and NLP problem.

## Real-World Applications

BoW and TF-IDF can be used in several traditional NLP Machine Learning applications.

### Spam Detection

Classify emails or messages as:

```
Spam
Not Spam
```

### Sentiment Analysis

Classify text as:

```
Positive
Negative
```

Neutral

## Text Classification

Classify articles into categories such as:

Sports  
Politics  
Technology  
Business  
Entertainment

## Search Systems

TF-IDF can help measure the importance of words in documents and support traditional information retrieval systems.

## Document Classification

Automatically classify documents based on their content.

## Final Takeaway

Natural Language Processing allows Machine Learning models to work with human language.

But Machine Learning algorithms cannot directly understand raw text.

Therefore, we need to convert text into numerical representations.

Two important traditional NLP vectorization techniques are:

Bag of Words  
↓  
Word Frequency  
↓  
Numerical Representation

and:

TF-IDF  
↓  
Term Frequency + Document Frequency  
↓  
Word Importance  
↓  
Numerical Representation

The key difference is:

**Bag of Words mainly focuses on how frequently words appear, while TF-IDF considers both word frequency within a document and how common the word is across documents.**

BoW is simple and useful for creating a basic text representation.

TF-IDF is useful when we want to give more importance to informative words and less importance to words that appear across many documents.

Both techniques provide a strong foundation for understanding traditional NLP and Machine Learning.



**Written by Marishetty Ramyakrishna**

0 followers · 2 following

Follow

